



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 07

NOMBRE COMPLETO: Sanchez Villalpando Johan

N° de Cuenta: 422028657

GRUPO DE LABORATORIO: 04

GRUPO DE TEORÍA: 02

SEMESTRE 2025-2

FECHA DE ENTREGA LÍMITE: 05/04/25

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Para el primer ejercicio se toma como partida el helicóptero que está en la práctica, se debe mover hacia adelante y hacia atrás, para ello se tomaron las variables de movimiento que se usaron en el automóvil, siendo articulación4, la cual permite moverse hacia adelante y hacia atrás

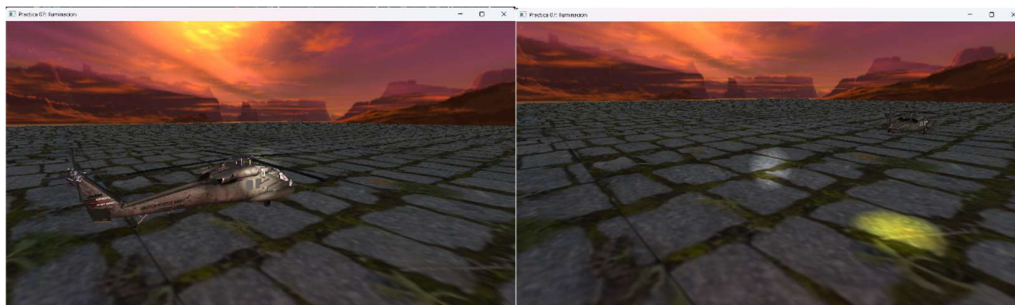
```
//helicoptero
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f - mainWindow.getarticulacion4(), 5.0f, 6.0f));
spotLights[1].SetPos(glm::vec3(0.0f - mainWindow.getarticulacion4(), 5.0f, 6.0f)); //luz
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

if (key == GLFW_KEY_K)
{
    theWindow->articulacion4 += 0.2;
    //theWindow->articulacion3 += 10.0; //lantas giran cuando avanza
}
if (key == GLFW_KEY_L)
{
    theWindow->articulacion4 -= 0.2;
    //theWindow->articulacion3 -= 10.0; //lantas giran cuando avanza
}

Window::Window(GLint windowHeight)
{
    width = windowHeight;
    height = windowHeight;
    rotax = 0.0f;
    rotay = 0.0f;
    rotaz = 0.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
}

GLfloat getarticulacion4() { return articulacion4; }
```

Podemos ver el movimiento usando K para adelante y L para atrás.



Para el segundo ejercicio fue necesario añadir una luz amarilla que siga al helicóptero, simulando la luz de algunos helicópteros.

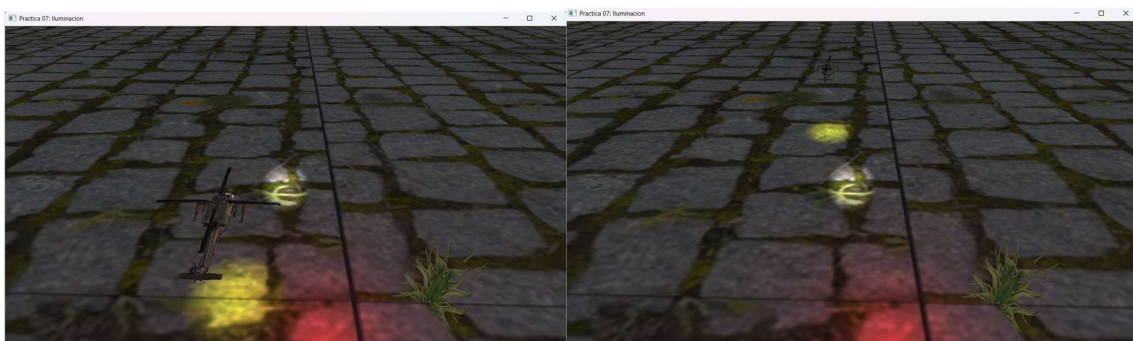
Para ello reciclé la luz azul del ejercicio y la modifiqué para hacerla amarilla, aumenté la radiación y la ajusté de modo que parezca la luz de un helicóptero de policía por ejemplo.

```
//luz fija color amarilla
spotLights[1] = SpotLight(1.0f, 1.0f, 0.0f,
    0.5f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, -1.0f, 0.0f, //dir
    1.0f, 0.0f, 0.0f,
    25.0f); //ángulo
spotLightCount++;
```

Después se añade al helicóptero usando la función SetPos, donde también se modifica usando la variable de movimiento del helicóptero, de modo que sigue el mismo movimiento.

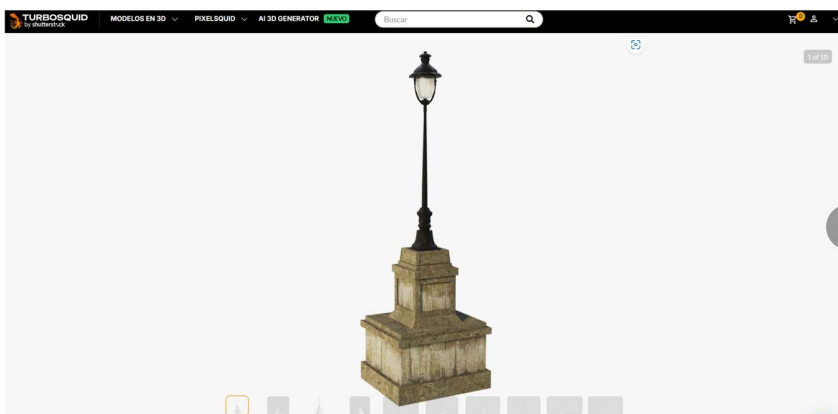
```
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f - mainWindow.getarticulacion4(), 5.0f, 6.0f));
spotLights[1].SetPos(glm::vec3(0.0f - mainWindow.getarticulacion4(), 5.0f, 6.0f)); //luz
```

Imagen del helicóptero con luz alejándose.



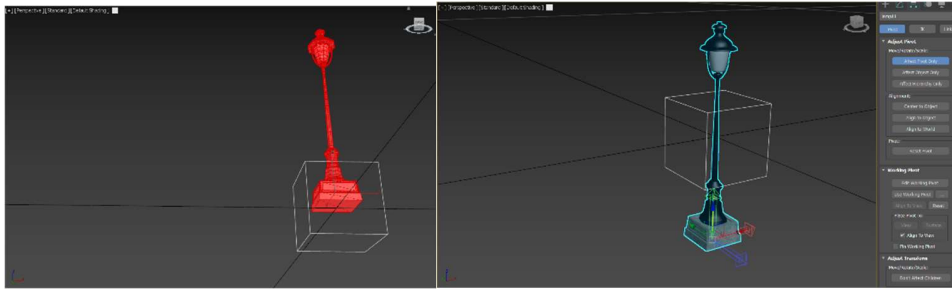
Para el tercer ejercicio se tiene que utilizar el modelo de una lámpara y añadirle luz puntual para que parezca que está iluminada.

Yo elegí el siguiente modelo:



La referencia se añade en la bibliografía.

Primero verifiqué que el modelo estuviera triangulizado y le hice una modificación para quitar la base y hacerlo más similar a un faro normal, también ajusté el pivote a la base de la lámpara.



Despues se añade el modelo.

```
//lampara
Model lampara;

//lampara
lampara = Model(); //Creamos modelo
lampara.LoadModel("Models/lampara.obj");
```

También se crea la spotlight blanca que será la luz que emitirá el faro desde el foco, la posición puede añadirse desde la creación de la luz, sin embargo preferí usar un SetPos junto a la lámpara por si quiero moverla de posición.

Se le pone radicación alta y un ángulo mayor para asemejar a una lampara real.

```
//luz fija color blanco lampara
spotLights[2] = SpotLight(1.0f, 1.0f, 1.0f,
    0.7f, 0.3f,
    0.0f, 0.0f, 0.0f, //pos
    0.0f, -1.0f, 0.0f, //dir
    1.0f, 0.0f, 0.0f,
    50.0f); //ángulo
spotLightCount++;
```

Despues se usa el modelo y se añade la spotlight, modificando la posición con SetPos.

```
//lampara
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(5.0f, -1.0f, -2.5f));
spotLights[2].SetPos(glm::vec3(5.0f, 1.25f, -2.5f)); //luz
model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
lampara.RenderModel();
```

Para tener una guía de la altura a la que posicionar la luz utilicé un modelo temporal para referencia.



Finalmente se obtiene la lampara con la luz en el foco.

Imagen junto a otra lampara apagada.



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

Para esta práctica no hubo complicaciones, las luces hasta ahora no son tan complicadas de manejar, lo más tardado fue encontrar un modelo de lampara gratis que fuera similar a lo que quería.

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.
- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Esta práctica fue sencilla, ya que retomamos cosas desde prácticas anteriores, como el movimiento con variables, el uso de modelos en 3dsmax las texturas, etc, además de que las luces hasta ahora han sido sencillas de manejar, la función SetPos fue muy útil.

La explicación fue buena y entendí el uso de las luces y cómo se generan. También pienso en el proyecto y todas las luces que debe tener, además de si aumentará mucho el procesamiento necesario para ejecutarlo y me doy unas ideas sobre las luces que se usarán y en qué partes.

Bibliografía en formato APA

- Marc Mons. (15 Sep 2024). *Calle de la vieja farola modelo 3d*. Turbosquid.com. Recuperado el 01 de abril de 2025, de <https://www.turbosquid.com/es/3d-models/3d-old-lamp-street-model-2281618#>